

MANUAL D COMANDO

Trinnity Viseron System v5.0 — Comandos Viseron
Integradas

yt-dlp

Ollama

Foocus

Whisper

Plausible

AppFlowy

Penpot

CUDACyclone

DATA · 09/08/2026

www.trinnityviseronssystem.io

TVS v5.0

SUMARIO

- [%_ 1. Comandos Viseron \(npm + CLI + API\) — lido de package.json](#)
- [%¹ 2. tvs_ytdlp — YouTube/Video Downloader](#)
- [%_ 3. tvs_ollama — IA Local \(LLaMA, Qwen, Mistral...\)](#)
- [%¹ 4. tvs_foocus — Gerador de Imagens AI](#)
- [%_ 5. tvs_whisper — Audio para Texto](#)
- [%¹ 6. tvs_plausible — Web Analytics](#)
- [%_ 7. tvs_appflowy — Workspace AI](#)
- [%¹ 8. tvs_penpot — Design Tool + MCP](#)
- [%_ 9. tvs_cudacyclone — GPU Bitcoin Puzzle Solver \(CUDA\)](#)
- [%¹ 10. API TVS Completa \(auth/billing/onboarding/email\)](#)
- [%_ 11. Como emitir comandos via Diretivas](#)

1 Comandos Viseron

Comandos npm para operar o Trinnity Viseron System (lidos automaticamente de package.json — novos comandos aparecem aqui a cada regeneração):

npm run start

Executar sistema compilado (dist)

npm run start:exe

Executar executável standalone Windows

npm run dev

Modo desenvolvimento com hot reload

npm run build

Compilar TypeScript para dist/

npm run test

Rodar testes core + web (67)

npm run test:core

Rodar testes do núcleo

npm run test:web

Rodar testes da camada web (auth/billing/onboarding/email/messaging)

npm run demo

Demo operacional real (HTTP 9 endpoints)

npm run test:hyper

Rodar testes hyperbrain

npm run build:android

Gerar APK Android

npm run build:ios

Gerar IPA iOS

npm run build:all

Gerar APK + IPA

npm run build:eas-android

Build Android via EAS

npm run build:eas-ios

Build iOS via EAS

npm run mobile:start

Iniciar app mobile Expo

npm run mobile:android

Executar app Android

npm run mobile:ios

Executar app iOS

npm run launch

Script de lançamento de mercado

npm run setup

Instalar todas as dependências

npm run lint

Verificar TypeScript (tsc --noEmit)

npm run backup

Executar backup diário

npm run backup:schedule

Agendar backup automático (Task Scheduler)

npm run skills

Skills CLI (list, search, info)

npm run skills:install

Instalar/atualizar coleções de skills (autônomo)

npm run skills:list

Listar skills instaladas

npm run skills:search

Pesquisar skills

npm run skills:info

Ver info de uma skill

npm run init

Build + backup + start

npm run init:full

Inicialização completa do sistema

npm run deploy

Deploy completo (build + PDFs + backup)

npm run deploy:github

Deploy GitHub

npm run deploy:render

Deploy Render via API

npm run deploy:hostalia

Deploy landing page via FTP Hostalia

npm run deploy:vercel

Deploy Vercel

npm run start:web

Executar servidor web compilado

npm run dev:web

Servidor web em modo desenvolvimento

npm run pitch

Gerar pitch de investidores

npm run pitch:startup

Gerar pitch startup (3 idiomas)

npm run pitch:v6

Gerar pitch v6

npm run roadmap

Gerar roadmap milionário

npm run docs:100

Gerar data/Viseron_100_Melhorias_Integracao.pdf

npm run report:update

Gerar relatório de update PDF

npm run report:state

Gerar relatório de estado PDF (o que podes fazer + estado real)

npm run docs:revenue

Gerar plano de receita real PDF (passo a passo Stripe/Gmail/domínio + modelo de receita)

npm run audit:arkom

Auditoria operacional ARKOM/AIOX (scan/diagnóstico/fix/verdicto !' Viseron_Audit_ARKOM.pdf)

npm run demo:jarvis

Demo do JARVIS (conversa + autonomia real sobre o sistema)

npm run gmail:setup

Setup Gmail API (OAuth consent !' refresh token)

npm run demo:email

Demo dos 3+ fluxos de email (verify/reset/invoice/agent)

npm run demo:messaging

Demo de mensageria E2E (contactos/conversas/grupos/leitura)

npm run pdfs:all

Regenerar TODOS os PDFs (manuais, pitches, roadmap, 100 melhorias)

npm run deploy:domain

Configurar domínio www.trinnityviseronssystem.io no .env

npm run deploy:domain:check

Validar DNS/HTTPS do domínio novo

npm run update:auto

Self-update: pull + install + PDFs + build + testes + deploy

npm run restart

scripts/restart.ps1

npm run test:omega

tests/omega.test.ts

npm run test:os

tests/os.test.ts

npm run test:restart

tests/restart.test.ts

npm run tvs

scripts/tvs.ts

npm run tvs:status

scripts/tvs.ts status

npm run tvs:list

scripts/tvs.ts list

npm run tvs:install

scripts/tvs.ts install

npm run tvs:uninstall

scripts/tvs.ts uninstall

npm run tvs:update

scripts/tvs.ts update

npm run tvs:doctor

scripts/tvs.ts doctor

npm run build:apk-installer

scripts/build-apk-installer.ps1

npm run omniroute:install

```
npm install omniroute
```

```
npm run omniroute:start
```

```
node -e "const{OmniRouteBridge}=require('./dist/src/integrations/omniroute/OmniRouteBridge');new OmniRouteBridge(null,{port:20128,autoStart:true}).initialize()"
```

npm run build:exe

```
node scripts/build-standalone.mjs
```

npm run build:exe:win

```
node scripts/build-standalone.mjs --platform win
```

npm run build:exe:mac

```
node scripts/build-standalone.mjs --platform mac
```

npm run build:exe:linux

```
node scripts/build-standalone.mjs --platform linux
```

npm run build:exe:all

```
node scripts/build-standalone.mjs --platform all
```

npm run build:electron

```
node scripts/build-standalone.mjs --platform win --electron
```

npm run build:electron:all

```
node scripts/build-standalone.mjs --platform all --electron
```

npm run electron:setup

```
cd electron && npm install
```

npm run electron:start

```
cd electron && npx electron .
```

npm run electron:build:win

```
cd electron && npm run build:win
```

npm run electron:build:mac

```
cd electron && npm run build:mac
```

npm run electron:build:linux

```
cd electron && npm run build:linux
```

npm run call:start

```
node -e "require('./dist/src/integrations/call-system/CallSystemBridge').startServer()"
```

npm run jarvis:start

```
node -e "require('./dist/src/integrations/openjarvis/OpenJarvisBridge').startServer()"
```

npm run asno:start

```
node -e "require('./dist/src/integrations/asno/ASNOBridge').startServer()"
```

npm run super:start

```
src/index.ts
```

npm run deploy:all

```
scripts/deploy-all.ps1 -Full
```

npm run deploy:full

```
scripts/deploy-all.ps1 -Full
```

npm run game:viseron

```
python tools/viseron-game/windows/viserongame.py
```

npm run game:viseron:demo

```
python tools/viseron-game/windows/viserongame.py --demo 30
```

npm run game:dos

tools/viseron-game/dos/start-dosbox.bat

npm run game:web

node dist/src/web/standalone-server.js

npm run game:apk

```
powershell -ExecutionPolicy Bypass -Command "if (Test-Path 'mobile/apps/viserongame/android') { Push-Location 'mobile/apps/viserongame/android'; .\gradlew.bat --no-daemon assembleRelease; Pop-Location } else { Push-Location 'mobile/apps/viserongame'; npx expo prebuild --platform android --no-install; Pop-Location; Push-Location 'mobile/apps/viserongame/android'; .\gradlew.bat --no-daemon assembleRelease; Pop-Location }; Copy-Item 'mobile/apps/viserongame/android/app/build/outputs/apk/release/app-release.apk' 'data/apps/viserongame.apk' -Force; Write-Host 'APK: data/apps/viserongame.apk'"
```

npm run cosmos:whitepaper

scripts/gerar-whitepaper-cosmos.ts

npm run cosmos:marketing

scripts/gerar-kit-marketing-cosmos.ts

npm run cosmos:contracts

scripts/gerar-cosmos-contratos.ts

npm run cosmos:logos

node scripts/gerar-logos-cosmos.mjs

npm run cosmos:bot

scripts/cosmos-telegram-bot.ts

npm run cosmos:kit

npm run cosmos:whitepaper && npm run cosmos:marketing && npm run cosmos:contracts && npm run cosmos:logos

npm run cosmos:golive

scripts/gerar-cosmos-go-live.ts

npm run cosmos:ops

scripts/gerar-cosmos-operacoes-wallet.ts

npm run cosmos:solana

node contracts/solana-go-live.mjs

npm run cosmos:wallet

node contracts/solana-wallet-generate.mjs

npm run cosmos:wallet:acesso

node contracts/solana-access-doc.mjs

npm run cosmos:wallets

node contracts/solana-wallet-factory.mjs

npm run cosmos:governanca

scripts/gerar-governanca-biblica.ts

npm run atlas:plan

scripts/gerar-plan-ingles-atlas.ts

npm run fama:instagram

scripts/gerar-plano-fama-instagram.ts

npm run omega:plan

scripts/gerar-omega-master-plan.ts

npm run audit:full

node scripts/full-audit.mjs

npm run report:evolution

scripts/gerar-relatorio-evolucao.ts

npm run squad:scan

src/omega/squads/SquadScanner.ts

npm run pdf:comandos

scripts/gerar-pdf-comandos-senhas.ts

npm run pdf:ficheiro

scripts/gerar-ficheiro-projeto.ts

npm run pdf:all-trilingual

scripts/gerar-pdf-comandos-senhas.ts && tsx scripts/gerar-ficheiro-projeto.ts && tsx scripts/gerar-pitch-investidores.ts && tsx scripts/gerar-pitch-startup.ts

npm run cudacyclone

scripts/cudacyclone.ts

npm run domain:check

scripts/setup-dominio.ps1 -Validate

npm run domain:novo

scripts/setup-dominio-novo.ps1

npm run domain:novo:check

scripts/setup-dominio-novo.ps1 -Validate

npm run go-live:stripe

scripts/go-live-stripe.ts

npm run cofre

scripts/gerar-cofre-credenciais.ts

npm run plan:strategic

scripts/gerar-plano-estrategico.ts

npm run audit:completa

scripts/gerar-auditoria-completa.ts

npm run models:pull

ollama pull qwen2.5:3b && ollama pull qwen2.5:1.5b

npm run demo:avirato

scripts/demo-avirato.ts

npm run composio:status

scripts/composio.ts status

npm run composio:connect

scripts/composio.ts connect

npm run composio:connect-apps

scripts/composio.ts connect-apps

npm run composio:list-apps

scripts/composio.ts list-apps

npm run contas:pdf

scripts/gerar-relatorio-contas.ts

npm run expansion:pdf

scripts/gerar-registo-expansao.ts

npm run plano:agencia

scripts/gerar-plano-agencia-viseron.ts

npm run agency:demo

scripts/demo-agency.ts

npm run app:create

scripts/criar-app.ts

npm run app:build

scripts/criar-app.ts --build

npm run app:derecho

scripts/criar-app-derecho.ts

npm run rcs

scripts/rcs.ts

npm run rcs:status

scripts/rcs.ts status

npm run rcs:send

scripts/rcs.ts send

npm run rcs:list

scripts/rcs.ts list

npm run import:telecom

scripts/import-telecom.ts

npm run telecom:campaign

scripts/campaign-telecom.ts

2 yt-dlp — Downloader Universal

Repositorio: github.com/yt-dlp/yt-dlp

Baixa videos/audios de YouTube, Twitter, TikTok, Instagram, Twitch, Facebook e 1000+ sites.

pip install yt-dlp

Instalacao

Comandos Viseron:

tv�_ytdlp url='https://youtube.com/watch?v=...' modo=video qualidade=best

Baixar video na melhor qualidade

tv�_ytdlp url='...' modo=audio

Baixar apenas audio (MP3)

tv�_ytdlp url='...' modo=video qualidade=4k

Baixar video em 4K

tv�_ytdlp url='...' modo=video playlist=no

Baixar apenas um video (nao playlist)

tv�_ytdlp url='...' legendas=true lang='pt,en'

Baixar com legendas

tv�_ytdlp url='...' dir='./meus-videos'

Salvar em diretorio especifico

3 Ollama — IA Local

Repositorio: github.com/ollama/ollama

Executa modelos de linguagem localmente: LLaMA, Qwen, Mistral, DeepSeek, Phi, Gemma.

ja instalado (ollama v0.32.5)

Instalacao

Comandos Viseron:

tvsv_ollama acao=listar

Listar modelos disponiveis localmente

tvsv_ollama acao=puxar modelo='llama3.2'

Baixar modelo LLaMA 3.2

tvsv_ollama acao=chat modelo='llama3.2' prompt='O que e IA?'

Perguntar ao modelo local

tvsv_ollama acao=embed modelo='llama3.2' prompt='texto'

Gerar embedding de texto

tvsv_ollama acao=remover modelo='modelo-antigo'

Remover modelo baixado

ollama run deepseek-r1:7b

Comando direto: rodar DeepSeek R1

Repositorio: github.com/lillyasviel/Fooocus

Gera imagens fotorrealistas com IA, estilo Midjourney, sem necessidade de engenharia de prompt.

```
git clone https://github.com/lillyasviel/Fooocus.git && pip install -r requirements.txt
```

Instalacao

Comandos Viseron:

```
tvsv_fooocus acao=gerar prompt='gato astronauta' estilo=fantasy
```

Gerar imagem com descricao

```
tvsv_fooocus acao=gerar prompt='...' negativo='borrado, feio'
```

Gerar com prompt negativo

```
tvsv_fooocus acao=gerar prompt='...' passos=50 largura=1920 altura=1080
```

Imagem HD com mais qualidade

```
tvsv_fooocus acao=status
```

Verificar status do servidor Fooocus

```
cd ../Fooocus && python entry_with_update.py
```

Iniciar interface web (porta 7865)

5 Whisper — Audio para Texto

Repositorio: github.com/openai/whisper

Transcreve audio/video para texto com IA. Suporta 99+ idiomas. Modelos: tiny, base, small, medium, large, turbo.

```
pip install openai-whisper
```

Instalacao

Comandos Viseron:

```
tvw_whisper acao=transcrever arquivo='audio.mp3' modelo=medium
```

Transcrever audio em portugues

```
tvw_whisper acao=transcrever arquivo='video.mp4' modelo=large idioma=en
```

Transcrever video em ingles

```
tvw_whisper acao=transcrever arquivo='aula.wav' modelo=turbo formato=vtt
```

Transcrever e gerar legendas VTT

```
tvw_whisper acao=listar
```

Listar modelos disponiveis com descricao

```
python -m whisper audio.mp3 --model base --language pt
```

Comando direto Python

6 Plausible — Web Analytics

Repositorio: github.com/plausible/community-edition

Analytics web leve, open-source, respeitador de privacidade. Alternativa ao Google Analytics.

```
git clone https://github.com/plausible/community-edition.git && docker-compose up -d
```

Instalacao

Comandos Viseron:

```
tv�_plausible acao=deploy
```

Instrucoes para deploy com Docker

```
tv�_plausible acao=status
```

Verificar status do servidor Plausible

```
tv�_plausible acao=stats site='meusite.com' chave='API_KEY'
```

Obter estatísticas do site

```
git clone https://github.com/plausible/community-edition.git
```

Clonar repositório

```
docker-compose up -d
```

Iniciar com Docker (porta 8000)

Repositorio: github.com/AppFlowy-IO/AppFlowy

Workspace colaborativo open-source com IA integrada. Alternativa ao Notion e Monday.com.

```
git clone https://github.com/AppFlowy-IO/AppFlowy.git && docker-compose up -d
```

Instalacao

Comandos Viseron:

```
tv�_appflowy acao=deploy
```

Instrucoes para deploy Docker

```
tv�_appflowy acao=status
```

Verificar containers AppFlowy rodando

```
git clone https://github.com/AppFlowy-IO/AppFlowy.git
```

Clonar repositorio

```
docker-compose up -d
```

Iniciar servicos AppFlowy

8 Penpot — Design + MCP

Repositorio: github.com/penpot/penpot-mcp

Ferramenta de design open-source com protocolo MCP. Alternativa ao Figma com integracao AI.

```
git clone https://github.com/penpot/penpot.git && git clone https://github.com/penpot/penpot-mcp.git
```

Instalacao

Comandos Viseron:

```
tv�_penpot acao=deploy
```

Instrucoes para deploy Penpot + MCP

```
tv�_penpot acao=mcp
```

Configuracao do servidor MCP Penpot

```
tv�_penpot acao=exportar projeto='ID'
```

Exportar design para SVG/PNG/HTML

```
git clone https://github.com/penpot/penpot.git
```

Clonar repositorio principal

```
git clone https://github.com/penpot/penpot-mcp.git
```

Clonar repositorio MCP

Repositorio: github.com/Dookoo2/CUDACyclone

Solver de 'Satoshi puzzles' (busca de chave privada em intervalo) com aceleracao CUDA em GPU NVIDIA. Baseado em VanitySearch/Bitcrack. 6.5Gkeys/s (RTX 4090). Requer NVIDIA GPU + CUDA toolkit (Linux/WSL2).

```
git clone https://github.com/Dookoo2/CUDACyclone.git && make (com CUDA toolkit instalado)
```

Instalacao

Comandos Viseron:

```
tvsv_cudacyclone acao=status
```

Verificar binario, requisitos e instalacao

```
tvsv_cudacyclone acao=build
```

Clonar e compilar com make (GPU NVIDIA + CUDA)

```
tvsv_cudacyclone acao=run range='2000000000:3FFFFFFFFF'
address='1HBtApAFA9B2YZw3G2YKSMctb3dVnjuNe2' grid='512,256'
```

Buscar chave privada no intervalo

```
tvsv_cudacyclone acao=run range='...' target-hash160='...' grid='128,128' slices='16'
```

Buscar por hash160 (sem precisar de endereco)

```
tvsv_cudacyclone acao=benchmark
```

Velocidades por GPU (RTX 4060/4090/5090/3070)

```
./CUDACyclone --range 2000000000:3FFFFFFFFF --address 1HBtApAFA9B2YZw3G2YKSMctb3dVnjuNe2 --
grid 512,256
```

Comando direto (RTX 4060 ~1238 Mkeys/s)

```
./CUDACyclone --range 2000000000000:3fffffffffffff --address 1F3JRMWudBaj48EhwcHDdpeuy2jwACNxjP --
grid 128,128 --slices 16
```

Tune otimizado RTX 4090 (~6.1 Gkeys/s)

Endpoints REST para controlar o TVS remotamente:

GET /api/health

Health + db + billing + email + contagens

GET /api/metrics

Métricas de uso

GET /api/system/status

Status do servidor standalone

POST /api/waitlist

Registrar email na waitlist

GET /pitch/:file

Baixar PDFs de pitch

POST /api/auth/register

Registro multi-tenant (org != tenant + owner + JWT)

POST /api/auth/login

Login JWT (rate-limited)

GET /api/auth/me

Perfil autenticado

PATCH /api/auth/profile

Atualizar nome/perfil/role

GET /api/auth/users

Listar membros (owner/admin)

POST /api/auth/logout

Terminar sessão

GET /api/billing/plans

Planos Core \$29 / Pro \$99 / Enterprise \$499

POST /api/billing/checkout

Criar sessão de checkout

POST /api/billing/webhook

Webhook de pagamento !' upgrade do plano

GET /api/billing/subscription

Estado da subscrição/trial

GET /api/onboarding/templates

5 templates (conteúdo, atendimento, código, Squad AIOX, Arkom)

POST /api/onboarding/apply

Materializar agentes no workspace do tenant

GET /api/email/status

Provider de email ativo + estado Gmail

POST /api/email/test

Enviar email de teste

POST /api/email/verify/send

Enviar código de verificação

POST /api/email/verify/confirm

Confirmar código de verificação

GET /api/email/verified

Estado de verificação do email

POST /api/email/reset/send

Enviar código de reposição de password

POST /api/email/reset/confirm

Confirmar código + nova password

GET /api/messaging/status

Estado da mensageria + crypto (x25519/aes-256-gcm)

POST /api/messaging/key

Gerar/obter chave pública X25519 do utilizador

GET /api/messaging/contacts

Listar contactos

POST /api/messaging/contacts

Adicionar contacto por email

GET /api/messaging/conversations

Listar conversas + unread

POST /api/messaging/conversations

Criar conversa direta

POST /api/messaging/groups

Criar grupo

GET /api/messaging/conversations/:id/messages

Ler mensagens (desencriptadas para o membro)

POST /api/messaging/conversations/:id/messages

Enviar mensagem E2E (cifrada por recetor)

POST /api/messaging/conversations/:id/read

Marcar conversa como lida

GET /api/blog/posts

Listar posts do blog

POST /api/content/generate

Gerar post de conteúdo

POST /api/content/trigger

Disparar geração de conteúdo

GET /api/content/schedule

Estado do agendamento de conteúdo

GET /api/jarvis/status

Estado do JARVIS (provider, capacidades, autonomia)

POST /api/jarvis/chat

Conversar com o JARVIS (sessão + execução real de operações, rate-limited 30/min)

GET /api/revenue/readiness

Go-live de receita real: Stripe, webhook, Gmail, email provider, domínio, Postgres

Toda ferramenta no TVS é acessível via diretivas. O fluxo é:

- % 1. Agente decide qual ferramenta usar baseado na missão
- % 2. Agente chama `toolManager.executeTool('tvs_ytdlp', { url: '...' })`
- % 3. ToolManager executa o comando real no sistema
- % 4. Resultado retorna ao agente com sucesso/erro + dados
- % 5. Agente consolida resposta e retorna ao usuário

Exemplo de Diretiva:

```
POST /api/directive
{
  "id": "directive_yt_001",
  "objective": "Baixar vídeo do YouTube sobre IA",
  "squad": ["agent_mind_343_elon_musk"],
  "ratifiedBy": "trinnity-hurtado",
  "commandedBy": "pedro-costa",
  "budget": "100 VSR"
}
```

Exemplo de Chat Direto:

```
Agente: "Elon, baixa o último vídeo do Neuralink"
Elon: "tvs_ytdlp url=https://youtube.com/... modo=video"
Resultado: Vídeo baixado em ./data/downloads/
```